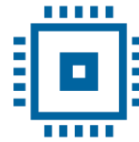




Transilvania
University
of Brasov



Transilvania
University
of Brasov

FACULTY OF ELECTRICAL ENGINEERING
AND COMPUTER SCIENCE

Department of Electronics and Computers

Study programme: Applied Electronics

Verification Project

Functional Verification

Environment for an Electric

Oven

Table of Contents

1	The digital design of the circuit.....	3
1.1	Block Diagram.....	3
1.2	Description of the general functionality of the DUT (Device Under Test).....	3
1.3	DUT Interfaces.....	5
1.4	List of DUT Functionalities.....	8
1.5	Waveforms.....	10
1.6	Register Table.....	10
2	Verification of the Digital Circuit Project.....	11
2.1	Architecture of the Verification Environment.....	11
2.1.1	APB Driver.....	12
2.1.2	Scoreboard / Reference Module.....	12
2.1.3	Door Driver.....	13
2.1.4	APB Monitor.....	13
2.1.5	Door Monitor.....	13
2.1.6	LED Interface Monitor.....	14
2.1.7	Door Transaction.....	14
2.1.8	APB Interface Transaction.....	14
2.1.9	LED Transaction.....	14
2.2	LIST OF VERIFICATION ELEMENTS.....	14
1.	Properties to Verify with Assertions.....	14
2.3	FUNCTIONAL COVERAGE ELEMENTS.....	15
2.4	Tests.....	16
2.5	Results Obtained.....	17

1 THE DIGITAL DESIGN OF THE CIRCUIT.

Block Diagram

General Functionality Description of the DUT (Device Under Test)

DUT Interfaces

List of DUT Functionalities

Schematic Diagram

Waveforms

Register Table

1.1 BLOCK DIAGRAM

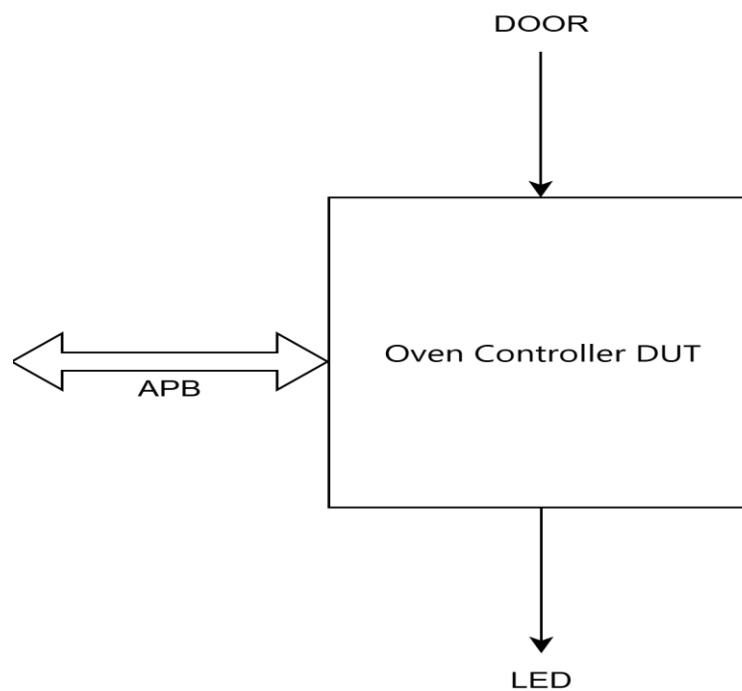


Figure 1. Interfaces used for the project

1.2 DESCRIPTION OF THE GENERAL FUNCTIONALITY OF THE DUT (DEVICE UNDER TEST)

For this project, we chose to test the functionality of an electric oven. Initially, we selected five key functions of the oven: temperature settings, a timer for tracking the duration of each function (which will be set by the user), and a LED that will turn on when the current task is done (when the timer reaches zero, the LED indicates that the cooking process is complete and the dish can be removed from the oven). The LED light is up only if the door is closed. When the door opens, the led turns off. Also, the LED is not turned on in the IDLE state.

We will now describe the functionality of our project: A person has an electric oven at their disposal and can use five main functions to prepare food:

1. **Heating:** For simple reheating of dishes.
2. **Cooking:** For standard cooking processes.
3. **Defrosting:** Specifically for defrosting foods such as vegetables and fruits.
4. **Pizza Function:** Dedicated to baking pizza.

After selecting the desired mode, the temperature must be configured. The oven supports five temperature settings mapped from the values 0 to 4, corresponding to 0°C, 50°C, 100°C, 150°C, and 200°C. The user then writes the cooking duration into the common timer register (timer_reg), which is shared by all operating modes. All processes have the same duration.

Important: The reset function is active on 1.

The oven includes a door sensor. In the DUT implementation, the door signal does not block the timer or the operating-mode selection; instead, it is used by the ready logic, so the LED can turn on only when the door is closed.

If the user wants to disable the ready indication before timeout, the operating mode can be returned to IDLE and/or the temperature can be set to 0.

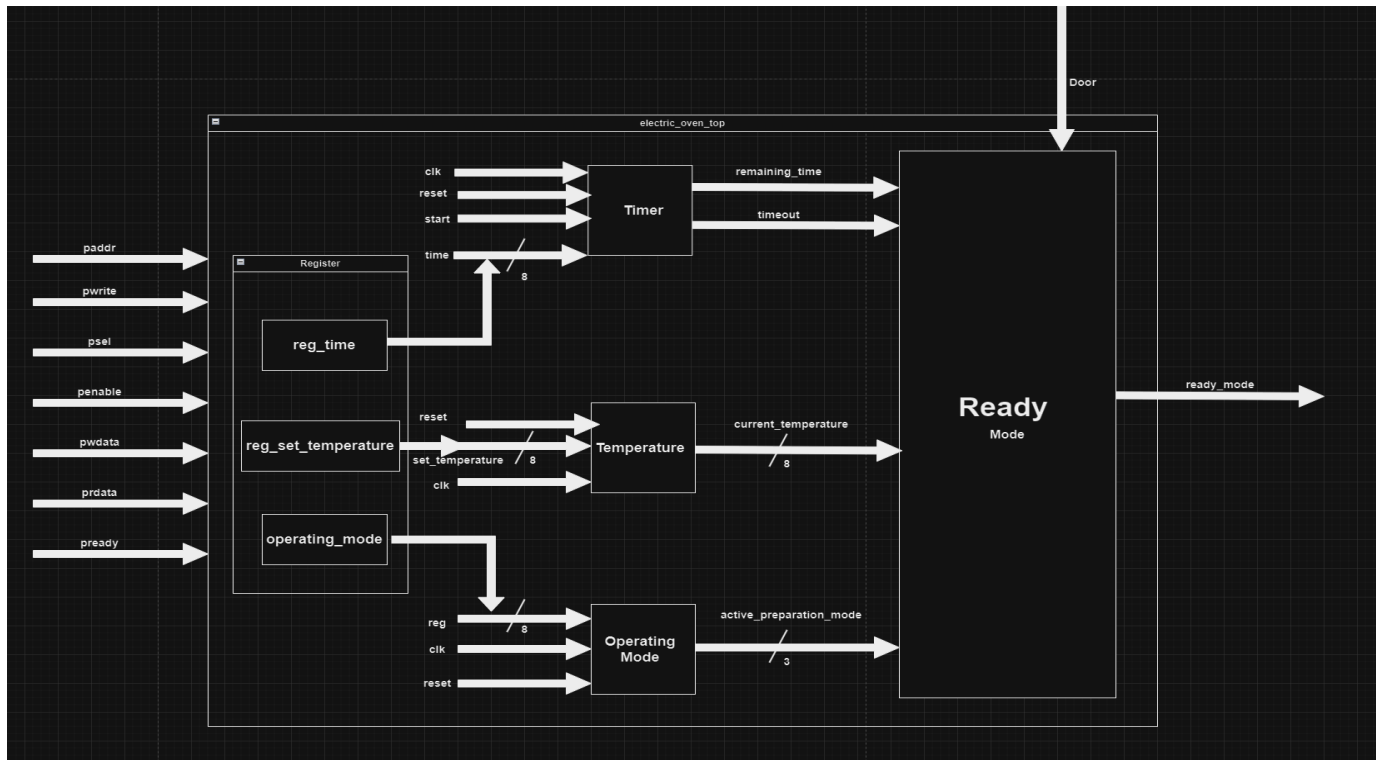


Figure 2. DUT Schematic

The countdown process starts when bit 3 of operation_mod_reg is set to 1. This start bit is automatically cleared to 0 by hardware in the next clock cycle.

The LED turns off when any of the following conditions are no longer met: the operating mode becomes IDLE, the temperature is set to 0, the timer has not yet expired, or the door is opened.

1.3 DUT INTERFACES

Table 1. Signals in the data transmission interface

APB INTERFACE			
Signal Name	Number of Bits	Direction	Description
clk	1	Input	Clock signal

reset	1	Input	Reset signal (active high — asserted when 1)
PADDR	ADDR_WIDTH	Output	Address signal, parameterizable up to 32 bits (currently 8 bits in this project).
PSELx	1	Output	Used for slave selection.
PENABLE	1	Output	Indicates the second and subsequent transfer cycles.
PWRITE	1	Output	Write access
PWDATA	DATA_WIDTH	Output	Write data bus when PWRITE is HIGH
PREADY	1	Input	The slave can accept data and address transfers
PRDATA	DATA_WIDTH	Input	Read data bus when PWRITE is LOW
PSLVERR (not used in this project)	1	Input	Optional signal in case of transfer errors

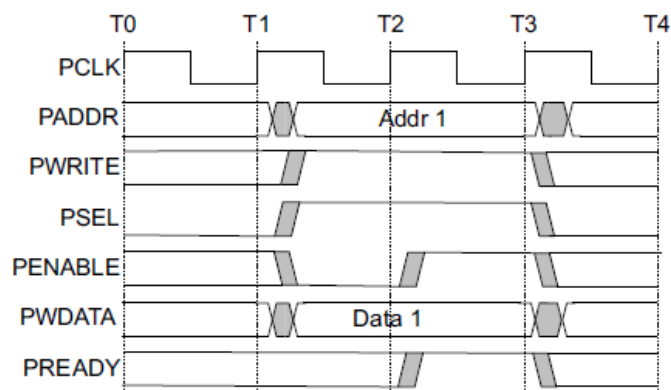


Fig. 3: APB Protocol Signals for Writing

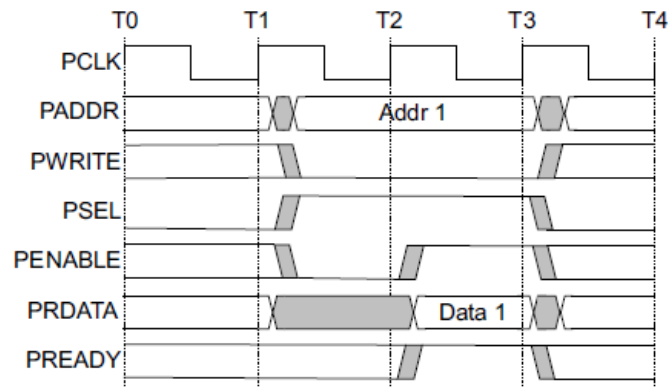


Fig. 4: APB Protocol Signals for Reading

Table 2. Signals in the Configuration Interface

Door INTERFACE			
Signal Name	Number of Bits	Direction	Description
door	1	Input	We indicate whether the oven door is closed or not. 1: door is closed 0: door is opened

Table 3. The signals within the display interface

The LED output interface			
led	1	Output	LED output — turns on when cooking is complete (operating mode active,

			temperature > 0, timer expired, door closed)
--	--	--	--

1.4 LIST OF DUT FUNCTIONALITIES

- The DUT models an electric oven that is configured through the APB interface. It supports operating-mode selection, temperature selection, and timer programming.
- The temperature is stored in `temp_set_reg`. Valid selector values are 0, 1, 2, 3, and 4, which map to 0°C, 50°C, 100°C, 150°C, and 200°C, respectively.
- The timer value is stored in `timer_reg` and can range from 0 to 255. Once started, the timer decrements by one on each clock cycle until it reaches 0, after which the timeout signal is asserted.
- The operating mode is stored in `operation_mod_reg[2:0]`, with the valid encodings `IDLE=3'b000`, `COOK=3'b001`, `HEAT=3'b010`, `PIZZA=3'b011`, and `DEFROST=3'b100`. Values greater than `3'b100` are ignored; the DUT does not implement a multi-mode priority scheme.
- The ready logic asserts the LED only when an operating mode is active, the converted temperature is greater than 0, timeout is asserted, and the door is closed.

Challenges Encountered

- At the beginning of the DUT implementation, instead of testing each module after its completion, We developed all the modules, including the top-level module with the testbench, and only performed a single test at the end with all the combined modules, which, understandably, did not work.
- After completing all the modules, We became stuck for a while on the timer module. It functioned correctly when compiled alone in ModelSim, but failed to operate when compiled with all the modules. After wasting several hours, we concluded that the module was correctly written; the issue lay in the testbench, where we had not allocated sufficient delay for the module.
- I had assumed that my colleague responsible for the APB interface was managing everything related to it, so I did not include it in the DUT. This oversight was brought to my attention later.

- When I began designing the DUT, I did not fully understand the concept of registers and that the design needed to be implemented with the APB interface in mind. As a result, I created a module that was less than ideal, with the implementation of the APB interface being added at the end of the process.

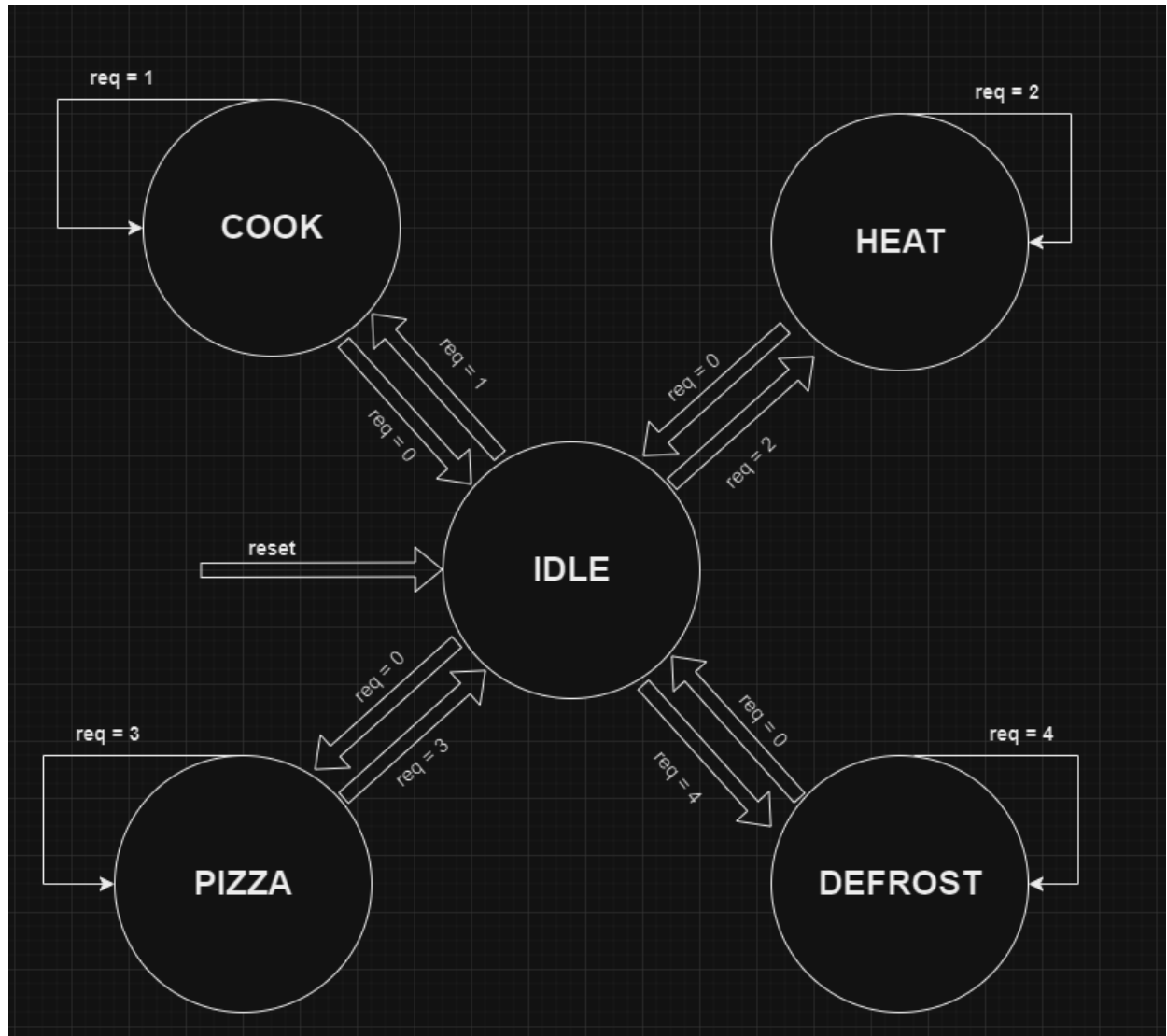


Figure 5. FSM Diagram of the Operating Mode Module

1.5 WAVEFORMS

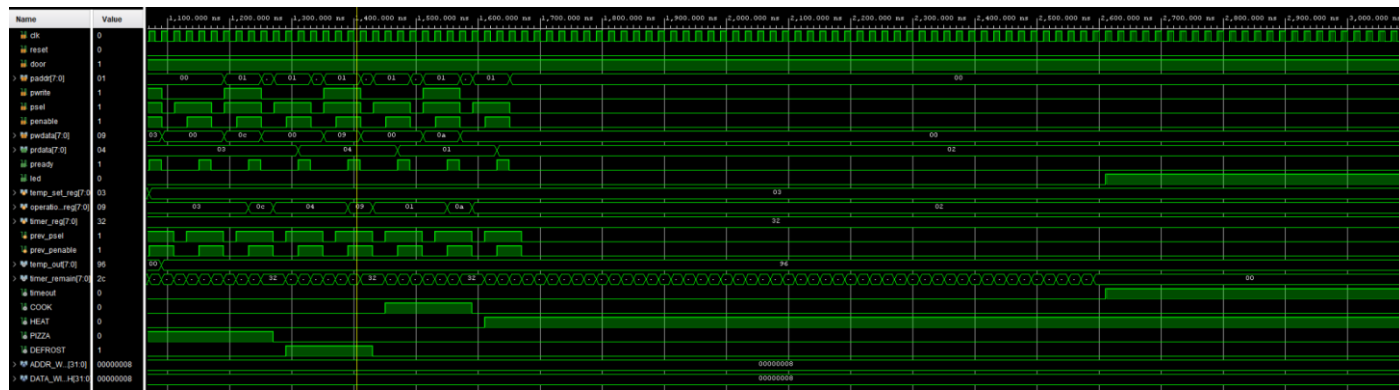
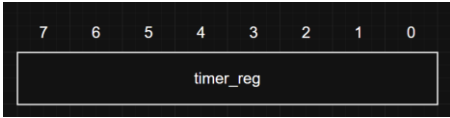


Figure 6. Resulted Waveforms in Vivado

1.6 REGISTER TABLE

Table 4. Table of Registers Accessible by the DUT

Register Name	Access Address	Register Structure	Description of Register Fields
temp_set_reg	0x00		The set temperature can have 5 values ranging from 0 to 4, where the temperature increases by 50 degrees starting from 0.
operation_mod_reg	0x01		This register indicates the mode being used and can take the following values: • IDLE = 3'b000 • COOK = 3'b001 • HEAT = 3'b010 • PIZZA = 3'b011 • DEFROST = 3'b100 A value bigger than 3'b100 in bits

			[2:0] of the operation_mod_reg is ignored Bit 3 is the start bit for the timer module and is used to initiate the countdown. This bit is automatically cleared to 0 in the next cycle after being written with 1. The typical sequence is to program timer_reg first and then write bit 3 to 1 in operation_mod_reg.
timer_reg	0x02		This register is used to set the timer value for the oven, which can range from 0 to 255.

2 VERIFICATION OF THE DIGITAL CIRCUIT PROJECT

Architecture of the Verification Environment

List of Elements to be Verified

Functional Coverage Elements

Tests

Obtained Results

2.1 ARCHITECTURE OF THE VERIFICATION ENVIRONMENT

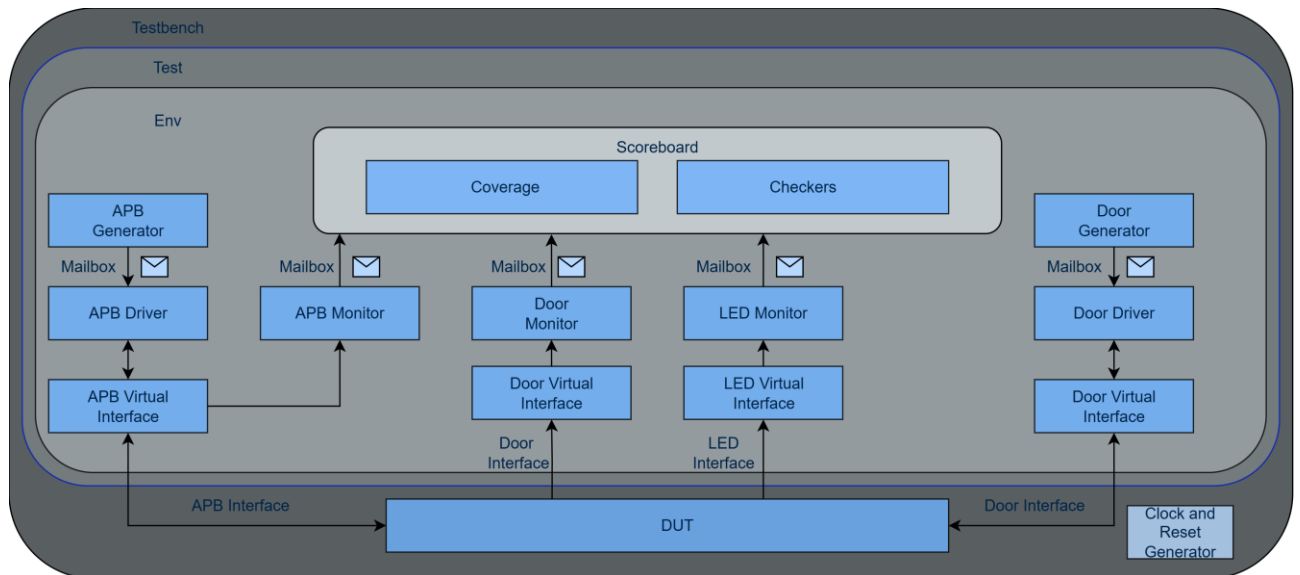


Figure 6. Overview of the components necessary for sending stimuli to the DUT via the APB interface

2.1.1 APB Driver

The APB driver retrieves data from the generator at an abstract level and sends it to the DUT according to the communication protocol for that interface. Here, the macro `APB_DRIV_IF` is declared, representing the interface that the driver will use to send data to the DUT. The reset requirements are also defined, initializing each signal. If the driver does not receive data from the generator, it remains in a waiting state until data is received.

2.1.2 Scoreboard / Reference Module

The scoreboard acts as the reference model for the DUT, predicting the expected LED output and comparing it against the actual DUT behavior in real time. It receives transactions from three monitors via dedicated mailboxes: the APB monitor (register writes and reads), the door monitor (door open/close events), and the LED monitor (actual LED state from the DUT).

Internally, the scoreboard maintains a set of shadow registers (`reg_timer`, `reg_config_apb`, `reg_config`, `reg_temperature`) that mirror the DUT's register file. `reg_config_apb` stores the raw APB write value for `operation_mod_reg`, while `reg_config` models the value observed after the hardware auto-clear of bit 3. When an APB write transaction is received, the corresponding shadow register is updated. When an APB read transaction is received, the scoreboard asserts that the read-back value matches the expected shadow register value.

The scoreboard implements several parallel tasks that communicate through shared state variables and synchronization events in order to accurately model the expected DUT behavior.

1. **collect_apb_transactions:** Receives APB transactions, updates shadow registers, and performs read-back assertions. It also converts the raw temperature selector (0–4) into actual degrees (0°–200°) using the same mapping as the DUT.
2. **collect_door_transactions:** Receives door transactions, updates the door state, and triggers a clock-cycle synchronization event (`one_clock_cycle`). After each door event, the start bit (bit 3) of the configuration register is cleared, modeling the DUT's hardware auto-clear behavior.
3. **counter_operation:** Models the DUT's countdown timer with a one-cycle pipeline delay (`timeout_pre` → `predicted_timeout`) to match the non-blocking assignment behavior of the RTL timer module. Signals completion via a chain event (`counter_done`).
4. **predict_led_output:** Waits on the `counter_done` event to guarantee it runs after `counter_operation` has updated the timeout state. Predicts the LED output using the same conditions as the `mod_ready` module: an active operating mode, temperature greater than zero, timeout asserted, and door closed.
5. **collect_led_transactions:** Receives the actual LED value from the LED monitor and compares it against the predicted value, incrementing match or mismatch counters accordingly.

At the end of the simulation, the scoreboard reports the total number of LED matches and mismatches.

2.1.3 Door Driver

The door driver collects transactions from the generator and transmits them to the DUT through an interface. This is accomplished via a virtual interface and a mailbox for receiving data from the generator. The driver operates in two modes: one that waits for the reset signal from the interface and another that sends transactions to the interface. The transmission process is contingent on receiving data from the generator and halts when the reset signal for the interface is activated.

2.1.4 APB Monitor

The APB monitor tracks data traffic on the DUT interfaces, from which verification data is gathered. In this implementation, the data from the interfaces are sent to the scoreboard for verification. The monitor declares and creates a transaction object that will contain the data retrieved from the interface.

2.1.5 Door Monitor

The door monitor tracks data traffic on the DUT's interfaces. It captures verification data and reconstructs the transactions. The data collected from the interfaces is sent to the scoreboard for verification. The monitor connects with the real DUT interface, collecting data and sending it to the scoreboard for verification.

2.1.6 LED Interface Monitor

The `led_monitor` class monitors data from the DUT interfaces and transmits it to a scoreboard for further processing. It can also be configured to collect coverage data associated with LED monitoring.

2.1.7 Door Transaction

The `door_transaction` class represents the data exchanged between the door generator and the door driver in the verification environment. The class contains the `door_closed` attribute, which stores the door state and receives random values through the `randomize()` function. The constraint models a 90% probability for the closed state (1) and a 10% probability for the open state (0). The `post_randomize` function displays the generated door state, and `do_copy` creates and returns a deep copy of the transaction object.

2.1.8 APB Interface Transaction

The `apb_transaction` class represents the data transferred between the APB generator, driver, and monitor. It contains the fields `paddr`, `pwrite`, `data`, and `delay`, and uses constraints to restrict the generated values. In the implementation, the `delay` field is stored in the transaction object but is not used by the APB driver to insert extra wait cycles between transfers. The monitor reconstructs APB transfers using objects of this class before forwarding them to the scoreboard.

2.1.9 LED Transaction

The `led_transaction` class represents the monitored LED state in the verification environment. It stores the `led` value observed at the DUT output and supports standard transaction utilities such as `post_randomize` and `do_copy`. This transaction is produced by the LED monitor and forwarded to the scoreboard for comparison against the predicted LED value.

2.2 LIST OF VERIFICATION ELEMENTS

Table 4. Put title here

1. Properties to Verify with Assertions	Description
---	-------------

psel_and_penable_active	Verify that after PSEL is asserted, PENABLE is asserted on the next clock cycle.
pready_after_psel_falls	Verify that when PSEL falls, PREADY also falls, marking the end of the transfer.
pwwdata_not_unknown_on_write	Verify that PWDATA is never X or Z during a write transfer.
pready_ends	Verify that PREADY remains asserted for only one clock cycle.
prdata_not_unknown_on_read	Verify that PRDATA is never X or Z during a completed read transfer (PSEL = 1, PENABLE = 1, PWRITE = 0, PREADY = 1).

2.3 FUNCTIONAL COVERAGE ELEMENTS

Three coverage groups are implemented, one for each DUT interface, to measure verification completeness.

Table 5. APB Functional Coverage Points

APB Coverage	
Cover Point	Description
pwrite_enable_cp	Both write transactions (pwrite = 1) and read transactions (pwrite = 0) have been used.
address_cp	All three valid register addresses have been accessed: temp_set_reg (0x00), operation_mod_reg (0x01), and timer_reg (0x02). Accesses to any other address are flagged as illegal_bins.
data_cp	The data bus values have been distributed across four equal

	ranges (range[0] = 0–63, range[1] = 64–127, range[2] = 128–191, range[3] = 192–255), and the boundary values have been hit (lowest_value = 0, highest_value = 255).
pwrite_enable_address_cx	Cross coverage between pwrite_enable_cp and address_cp, ensuring that each register has been both written and read.

Table 6. Door Interface Functional Coverage Points

Door Coverage	
Cover Point	Description
door_cp	Both door states have been used: door closed (door_closed = 1) and door open (door_closed = 0).

Table 7. LED Interface Functional Coverage Points

LED Coverage	
Cover Point	Description
led_cp	Both LED states have been observed at the DUT output: LED on (led = 1) and LED off (led = 0).

2.4 TESTS

Table 8. List of implemented tests for verifying the DUT (Device Under Test)

The name of the file that contains the test	Description of the test
random_test.sv	This is a small directed smoke test that writes two temperature values (50°C and 150°C) and selects the PIZZA mode before the environment

	is run.
operation_mod_test.sv	This test combines coverage-targeted APB accesses with a functional sequence that exercises all four operating modes (COOK, HEAT, PIZZA, and DEFROST), timer programming, and temperature configuration.
time_reg_test.sv	This test verifies timer_reg programming and read-back using a representative timer value, followed by a start command written through operation_mod_reg.
temperature_test.sv	This test verifies temperature changes using directed values (200°C, 150°C, and 50°C) while the DUT is configured through the APB interface.
reg_test.sv	This test first writes randomized values to the registers and then writes 0x55 (01010101) and 0xAA (10101010) to verify that each register bit can take both logic 0 and logic 1.
time_modop_test.sv	This test programs the operating mode, timer, and temperature registers together in order to exercise the scenario that can lead to the ready LED being asserted after timeout.

2.5 RESULTS OBTAINED

The verification environment was executed using the operation_mod_test.sv test, which includes both coverage-targeted transactions and a functional test sequence exercising all four operating modes (COOK, HEAT, PIZZA, DEFROST), temperature settings, and timer countdown operations.

The scoreboard reported 2499 LED matches and 0 mismatches, confirming that the DUT's LED output is correct under all tested scenarios.

Table 9. Functional Coverage Results Summary

Coverage Group	Cover Point	Result
APB Coverage	Address coverage	100.00%

APB Coverage	Type of transaction coverage (read/write)	100.00%
APB Coverage	Data coverage	100.00%
APB Coverage	Cross coverage (address × read/write)	100.00%
Door Coverage	Door state coverage	100.00%
LED Coverage	LED state coverage	100.00%

All coverage targets have been met at 100%, and no LED mismatches were detected during simulation.